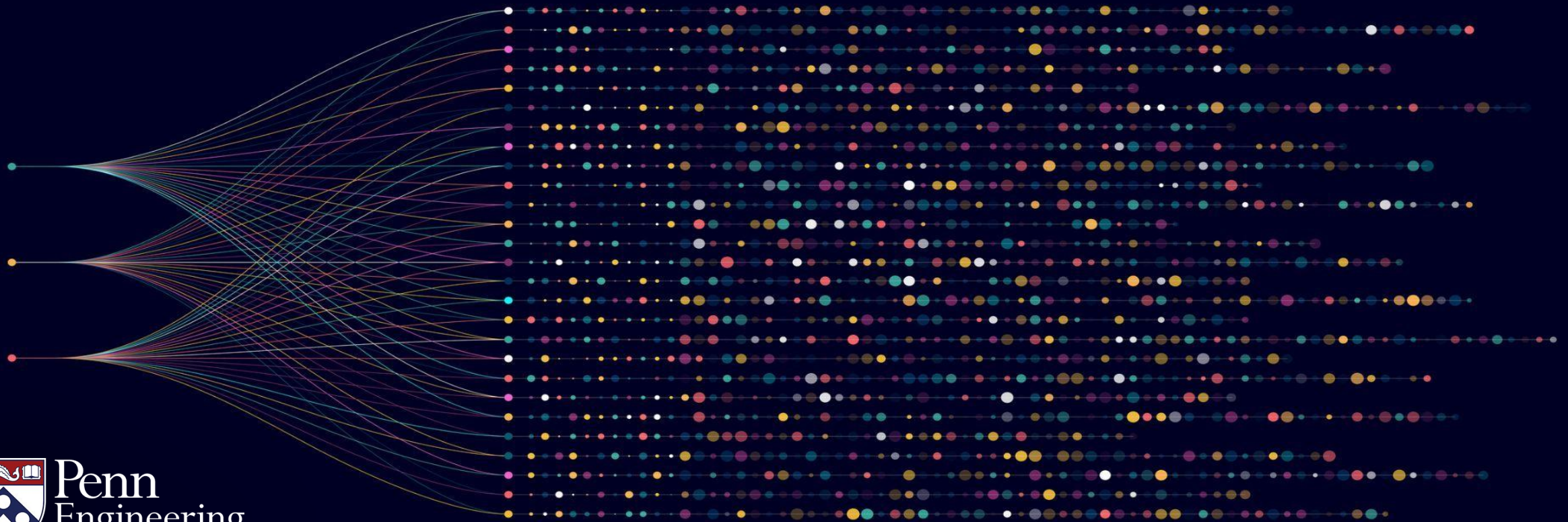


Algorithms for Big Data

Anindya De

Erik Waingarten

Estimating Averages: Problem Setup



Objectives: Computing Large Averages

- Motivating Scenario
- Defining the computational problem
- A “brute force” algorithmic benchmark

Motivating Scenario

- You are a software engineer working for a social network
- There is a (very large) database which holds all of user information and maintains usage information
- You are tasked with determining certain aggregate statistics on users and their usage information

Motivating Scenario - Example Questions

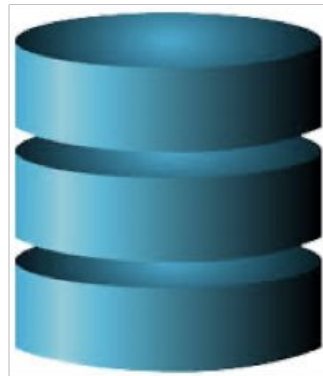
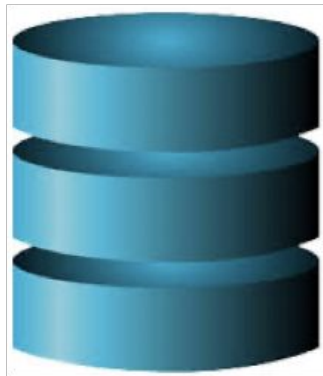
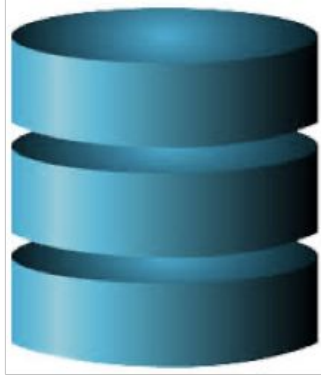
- How many users are in Europe?
- How many users posted in the past week?
- How long (on average) did users spend on the platform today?

Commonalities: Ask for large averages over the entire dataset

Outline for what is coming:

- First, provide background on the scenario
 - How data is stored
 - How data is accessed
 - What we want to do
- Then, define computational problem
 - Incorporating elements from the scenario
 - Establish algorithmic benchmarks

Background: What is the data?



Input data
(possibly in the cloud)

[illegible]

Rows may be users/events,
columns are attributes

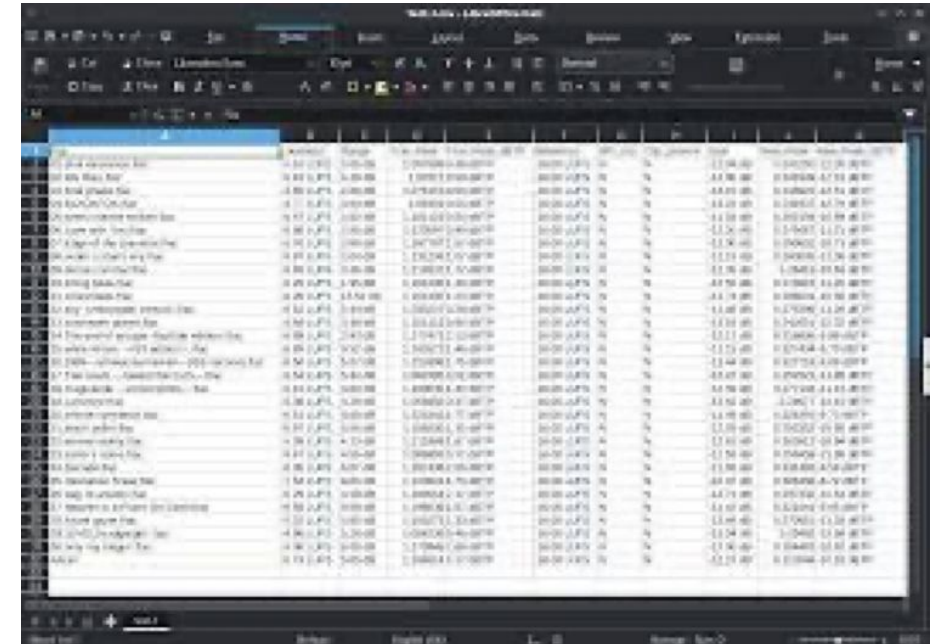
Background: What is the data?

Establish notation:

n = number of items.

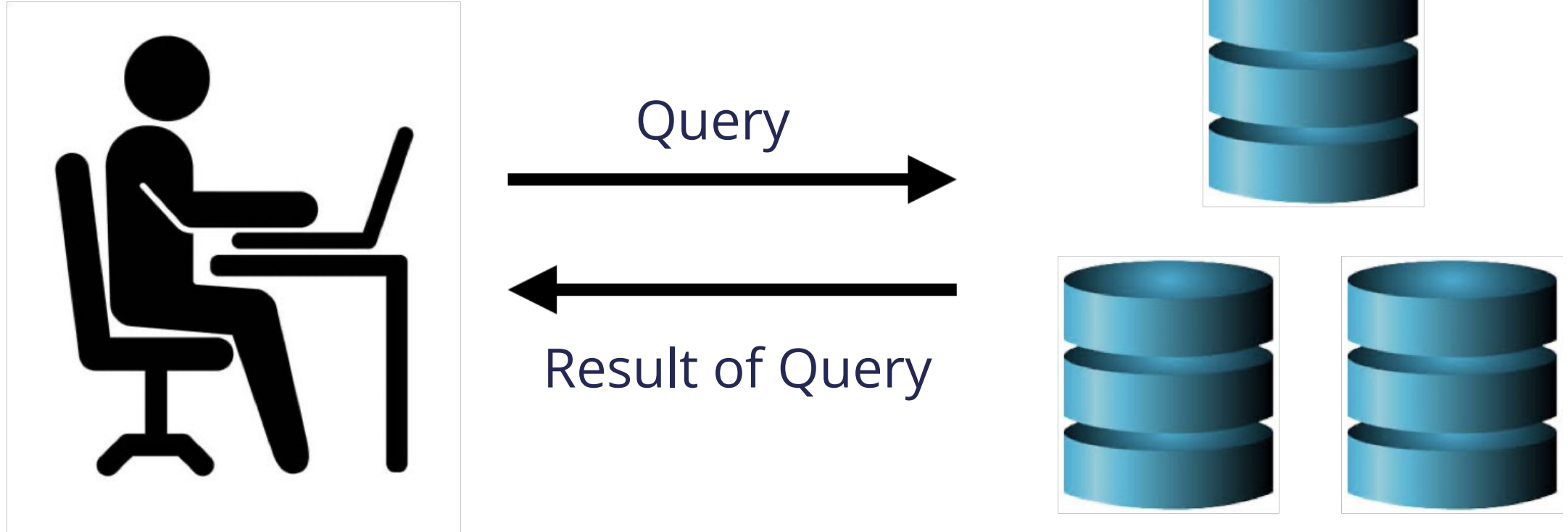
$$X_1, \dots, X_n \in \mathbb{R}$$

$$S = \frac{1}{n} \sum_{i=1}^n X_i$$

A screenshot of a data table, possibly from a spreadsheet application like Excel. The table has many columns and rows. The first column contains text labels, which appear to be names of items or events. The subsequent columns contain numerical values. The data is organized in a grid format with a header row and many data rows. The text is small, but the structure is clear as a tabular dataset.

Rows may be users/events,
columns are attributes

Background: What is the access?



Background: What is the access?

- The input data may not be readily accessible.
 - If data lies in a data center, communication costs may be bottleneck
- Input data is accessible. We want our algorithms to be very fast.
 - The running time of the algorithm is dominated by the number of queries made

Defining the Computational Problem.

Input: $X_1, \dots, X_n \in \mathbb{R}$

Goal: $S = \frac{1}{n} \sum_{i=1}^n X_i$

Queries: X_i

Examples of estimating large averages:

1. How many users are in Europe?
2. How many users posted in the past week?
3. How long did users spend on the platform on average?

Algorithmic Benchmark

Approach: brute force

Algorithm:

- Instantiate a variable which will maintain the sum
- Query every value and maintain sum
- Output the sum divided by n

Questions:

- Is it correct?
- How long should we expect it to take?

Can one be faster?

- Brute force: exact and deterministic
 - downside: slow
- Do there exist faster algorithms?
 - No, if we insist on exact or deterministic algorithms
- If we allow *approximation* and *randomization*, we can design significantly faster algorithms